Intermediate (chunking method changes the answer) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `chunking_practice.py`: the same document, split two different ways, can hand the model two different answers to the same question — this is the exercise that makes "chunking is a real design decision" something you watched happen, not something you took on faith. **If not:** the Build Task's chunker would be the first place a split fact ever bit you, in a pipeline with far more moving parts to debug through.

All examples below use the same test document and question — put them at the top of the `# Intermediate` section:

```
# chunking_practice.py — Intermediate section
PARAGRAPH_1 = (
    "Coffee brewing starts with the grind size. A coarse grind works best "
    "for a French press, because its metal filter lets fine grounds slip "
    "through and makes the coffee taste muddy and bitter. A fine grind is "
    "right for espresso instead, where water touches the grounds for only "
    "a few seconds."
)
PARAGRAPH_2 = (
    "Water temperature matters just as much as grind size. Water that is "
    "too hot, above 205 degrees Fahrenheit, scalds the coffee and pulls "
    "out bitter compounds that should have stayed in the grounds. Water "
    "that is too cool under-extracts the coffee, leaving it weak and sour "
    "even with plenty of grounds."
)
PARAGRAPH_3 = (
    "The final step is the pour itself. A slow, steady pour in a spiral "
    "pattern wets all the grounds evenly, which is why baristas practice "
    "just the pour for weeks before worrying about anything else."
)
# a blank line ("\n\n") between paragraphs, like a real text file
DOCUMENT = PARAGRAPH_1 + "\n\n" + PARAGRAPH_2 + "\n\n" + PARAGRAPH_3

QUESTION = "What water temperature should be avoided when brewing coffee?"
```

Basic Version

Approach 1 — the direct way

```
# chunking_practice.py — Intermediate section
import math
from openai import OpenAI

client = OpenAI()

def chunk_by_chars(text, chunk_size=200):
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i+chunk_size])
    return chunks

def chunk_by_paragraph(text):
    chunks = []
    for piece in text.split("\n\n"):
        stripped = piece.strip()
        if stripped:
            chunks.append(stripped)
    return chunks

def get_embedding(text):
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a, b):
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

def find_best_chunk(chunks, question_vector):
    best_chunk = None
```

```
    best_score = -1
    for chunk in chunks:
        chunk_vector = get_embedding(chunk)
        score = cosine_similarity(question_vector, chunk_vector)
        if score > best_score:
            best_score = score
            best_chunk = chunk
    return best_chunk, best_score

question_vector = get_embedding(QUESTION)

char_chunks = chunk_by_chars(DOCUMENT)
paragraph_chunks = chunk_by_paragraph(DOCUMENT)

char_best, char_score = find_best_chunk(char_chunks, question_vector)
paragraph_best, paragraph_score = find_best_chunk(
    paragraph_chunks, question_vector
)

print("chunk_by_chars best match, score:", char_score)
print(char_best)
print()
print("chunk_by_paragraph best match, score:", paragraph_score)
print(paragraph_best)
```

Expected output: with `chunk_size=200`, `chunk_by_chars` cuts this document right through the word "scalds" — one chunk ends `"...205 degrees Fahrenheit, sc"` and the next one starts `"alds the coffee..."`. Whichever half wins, it's missing part of the answer. `chunk_by_paragraph` keeps the whole "Water temperature matters..." paragraph — including "205 degrees Fahrenheit" and the reason why — as one unbroken chunk.

This version works correctly for what the exercise asks. It's missing type hints and reuses the same `get_embedding` call for every chunk on every run, with no reuse across calls.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — type hints and a `find_best_chunk` helper

```
# chunking_practice.py — Intermediate section
import math
from openai import OpenAI

client = OpenAI()

# DOCUMENT and QUESTION: the block at the top of this page


def chunk_by_chars(text: str, chunk_size: int = 200) -> list[str]:
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i + chunk_size])
    return chunks


def chunk_by_paragraph(text: str) -> list[str]:
    chunks = []
    for piece in text.split("\n\n"):
        stripped = piece.strip()
        if stripped:
            chunks.append(stripped)
    return chunks


def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding


def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
```

```
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)


def find_best_chunk(
    chunks: list[str], question_vector: list[float],
) -> tuple[str, float]:
    best_chunk = ""
    best_score = -1.0
    for chunk in chunks:
        chunk_vector = get_embedding(chunk)
        score = cosine_similarity(question_vector, chunk_vector)
        if score > best_score:
            best_score = score
            best_chunk = chunk
    return best_chunk, best_score


def main() -> None:
    question_vector = get_embedding(QUESTION)
    methods = {
        "chunk_by_chars": chunk_by_chars(DOCUMENT),
        "chunk_by_paragraph": chunk_by_paragraph(DOCUMENT),
    }
    for name, chunks in methods.items():
        best_chunk, best_score = find_best_chunk(chunks, question_vector)
        print(f"{name} best match, score: {best_score:.4f}")
        print(best_chunk)
        print()


if __name__ == "__main__":
    main()
```

Expected output: same conclusion as Basic — `chunk_by_paragraph`'s top match contains the whole temperature fact; `chunk_by_chars`'s top match is missing half of "scalds."

Difference from Basic: full type hints on every function. `find_best_chunk` has an explicit `tuple[str, float]` return type, so any caller knows exactly what comes back without reading its body. The two methods are driven from one `methods` dict and one loop in `main()`, instead of

copy-pasted print statements for each — adding a third chunking method later means adding one dict entry, not duplicating the comparison logic again.

Approach 2 — overlapping fixed-size chunks

Story: `chunk_by_chars` from Approach 1 can still cut a fact clean in half between two non-overlapping chunks — this is the actual fix, not a bigger `chunk_size` (which only moves the problem). **If not:** the `chunk_boundary_split` exercise's whole failure mode would still be sitting unfixed in this same file.

```
# chunking_practice.py — Intermediate section
def chunk_by_chars_with_overlap(
    text: str, chunk_size: int = 200, overlap: int = 40,
) -> list[str]:
    chunks: list[str] = []
    # why: less than chunk_size, so chunks share text
    step = chunk_size - overlap
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        i += step
    return chunks
```

Run this against `DOCUMENT` with `chunk_size=200, overlap=40` and look at the chunk boundaries: the word "scalds" that got split in half by the non-overlapping version now appears whole in at least one chunk, because each new chunk starts 40 characters *before* the previous one ended. The word that was previously exactly on the cut line is now safely inside the overlapping region instead of split by it.

Approach 3 — a paragraph chunker that doesn't silently give up

Story: `chunk_by_paragraph` trusts the document has real blank-line breaks — hand it one giant wall of text (a scraped page, a PDF that lost its formatting) and it silently returns the whole thing as "1 chunk," no error, no warning. **If not:** a search index that's secretly not chunked at all would look identical to a working one, until a search near the end of that one giant chunk came back oddly.

```
# chunking_practice.py — Intermediate section
def chunk_by_paragraph_safe(text: str) -> list[str]:
    pieces = chunk_by_paragraph(text)
    if len(pieces) <= 1:
        # how: no real paragraph breaks found -- fall back instead of
        # silently returning the whole document as "1 chunk"
        return chunk_by_chars_with_overlap(text, chunk_size=200, overlap=40)
    return pieces
```

```
# proof this matters: a document with no blank lines at all
single_block = DOCUMENT.replace("\n\n", " ")

naive_result = chunk_by_paragraph(single_block)
safe_result = chunk_by_paragraph_safe(single_block)

naive_count = len(naive_result)
safe_count = len(safe_result)
print("naive chunk_by_paragraph, no blank lines:", naive_count, "chunk(s)")
print("chunk_by_paragraph_safe, same document:", safe_count, "chunk(s)")
```

Expected output:

```
naive chunk_by_paragraph, no blank lines: 1 chunk(s)
chunk_by_paragraph_safe, same document: 5 chunk(s)
```

The plain `chunk_by_paragraph` hands back the entire document as a single "chunk" — no error, no warning, just a search index that's secretly not chunked at all. `chunk_by_paragraph_safe` notices that and falls back to the overlapping character chunker instead.

Approach 4 — the chunker as a parameter: `Callable[[str], list[str]]`

Story: Approaches 1–3 now give you four chunkers, and comparing them means calling each one by name. A function is a value in Python — you can pass `chunk_by_paragraph` (no `()`) into another function, which then calls it. **If not:** every new chunker would mean editing the comparison code, and the Build Task's `build_vector_store(doc_folder, chunk_fn)` — "the chunking method must be a swappable piece" — would be the first time you passed a function as an argument.

`Callable[[str], list[str]]` from `typing` is the type hint for "a function that takes one `str` and returns a `list[str]`" — exactly the shape of every chunker above.

```
# chunking_practice.py — Intermediate section
from typing import Callable


def best_chunk_for(
    chunk_fn: Callable[[str], list[str]],
    text: str,
    question_vector: list[float],
) -> tuple[str, float]:
    chunks = chunk_fn(text)
    return find_best_chunk(chunks, question_vector)
```

```
CHUNKERS: dict[str, Callable[[str], list[str]]] = {
    "chunk_by_chars": chunk_by_chars,
    "chunk_by_chars_with_overlap": chunk_by_chars_with_overlap,
    "chunk_by_paragraph": chunk_by_paragraph,
    "chunk_by_paragraph_safe": chunk_by_paragraph_safe,
}

question_vector = get_embedding(QUESTION)
for name, chunk_fn in CHUNKERS.items():
    chunk_count = len(chunk_fn(DOCUMENT))
    best_chunk, best_score = best_chunk_for(chunk_fn, DOCUMENT, question_vector)
    print(f"{name}: {chunk_count} chunks, best score {best_score:.4f}")
    print("   ", best_chunk[:60])
```

Expected output (the chunk counts are exact; scores and the chunk-based winners depend on the model, so yours will differ):

```
chunk_by_chars: 4 chunks, best score 0.54...
    ...
chunk_by_chars_with_overlap: 5 chunks, best score 0.5...
    ...
chunk_by_paragraph: 3 chunks, best score 0.55...
    Water temperature matters just as much as grind size. Water
chunk_by_paragraph_safe: 3 chunks, best score 0.55...
    Water temperature matters just as much as grind size. Water
```

`chunk_by_paragraph_safe` gives the same 3 chunks as `chunk_by_paragraph` here, because this `DOCUMENT` has real blank lines — its fallback only runs on a wall of text (Approach 3). `best_chunk_for` never names a chunker — it calls whatever `chunk_fn` it was given. Adding a fifth chunker is one new line in `CHUNKERS`.

Difference between Approaches 1–4: Approach 1's two chunkers both produce clean, non-overlapping pieces, and `chunk_by_paragraph` trusts that the document has real paragraph breaks to split on. Approach 2 fixes the boundary problem directly: overlap means a fact sitting near a cut usually survives whole in at least one chunk, without needing a bigger `k` at search time to compensate (the workaround `chunk_boundary_split`'s solution describes). Approach 3 fixes a different, quieter failure — a document that defeats `chunk_by_paragraph` entirely by having no blank lines — by detecting that case and falling back to Approach 2's chunker instead of returning one useless giant chunk. Approach 4 changes no chunker; it makes the chunker a parameter (`Callable[[str], list[str]]`), so any of them can be swapped in without touching the code that uses it.

Which one should you actually write? For a document set you've actually looked at (like this exercise's `DOCUMENT`), Approach 1's plain `chunk_by_paragraph` is fine — real paragraph breaks, no boundary problem to speak of. The moment your documents come from somewhere you don't fully control — scraped pages, converted PDFs, user uploads — reach for Approach 3's fallback, since you can't guarantee every file will have the blank-line structure you're assuming. Reach for Approach 2's overlap specifically for `chunk_by_chars`-style fixed-size chunking, any time a fact could plausibly land near a chunk boundary — which, over a large enough document set, it eventually will. Use Approach 4's `chunk_fn` parameter whenever more than one chunker exists — it's how the Build Task's `build_vector_store()` stays swappable.